CEBU INSTITUTE OF TECHNOLOGY
UNIVERSITY

IT342-G1
System Integration and Architecture

System Design Document (SDD)

Project Title: Creditful

Prepared By: Morales, Dilton Rowan Suico

Version: 0.4

Date: February 28, 2026
Status: Final

REVISION HISTORY TABLE

Version	Date	Author	Changes Made	Status
0.1	2/21/2026	Dilton Morales	Initial draft	Review
0.2	2/23/2026	Dilton Morales	Added API specifications	Review
0.3	2/27/2026	Dilton Morales	Updated database design	Review
0.4	2/28/2026	Dilton Morales	Added UI/UX designs	Draft
0.5	[Date]	Dilton Morales	Incorporated feedback	
0.6	[Date]	Dilton Morales	Final review and corrections	
1	[Date]	Dilton Morales	Baseline version for development	

TABLE OF CONTENTS

Contents

EXECUTIVE SUMMARY 4

1.0 INTRODUCTION 5

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION 5

3.0 NON-FUNCTIONAL REQUIREMENTS 10

4.0 SYSTEM ARCHITECTURE 12

5.0 API CONTRACT & COMMUNICATION 13

Authentication Endpoints 13

User Registration 13

User Login 14

Loan Management Endpoints (Examples) 15

Get Borrower Loan Balance 15

Create Loan (Admin Only) 15

6.0 DATABASE DESIGN 17

7.0 UI/UX DESIGN 19

8.0 PLAN 20

EXECUTIVE SUMMARY

1.1 Project Overview

Creditful is a loan management application designed for borrowers to monitor their personal loans and for public users to verify whether a borrower has an existing loan record. The system provides borrowers with access to their loan details, including principal amount, interest, total payable amount, remaining balance, payment history, and loan status. Public users can check whether a loan exists and its status without seeing sensitive financial information.

Creditful is built using Spring Boot for the backend API, React for the web interface, a mobile application for cross-platform access, and MySQL as the relational database. All components communicate through RESTful APIs to ensure secure, consistent, and real-time data access. The system also includes authentication, role-based access control, OAuth login, file uploads, email notifications, and optional real-time features, focusing on simplicity, transparency, and accessibility for all users.

1.2 Objectives

1. Develop a fully functional loan management system with authentication and security
2. Implement a three-tier architecture using Spring Boot (backend), React (web), mobile app, and MySQL
3. Allow borrowers to securely log in and view their loan details
4. Provide public loan verification with limited information
5. Implement role-based access control (Admin and Borrower)
6. Design a responsive interface that works consistently across web and mobile
7. Deploy all system components to production-ready environments

1.3 Scope

Included Features:

- User registration, login, logout, JWT authentication, password hashing (BCrypt), protected routes
- Role-based access control (Admin and Borrower)
- Borrower dashboard (own loans + limited view of other borrowers' loans)
- Payment history display
- Public loan verification (loan exists? ACTIVE/COMPLETED)
- Google OAuth login
- File upload linked to loan records (PDF or image)
- Email notifications (welcome/system)
- Sandbox payment gateway integration (if applicable)
- Responsive web interface
- Mobile application
- MySQL relational database with proper relationships

Excluded Features:

- Real payment gateway integration
- Credit score integration
- Advanced analytics dashboard
- Push notifications

1.0 INTRODUCTION

1.1 Purpose

This document serves as the comprehensive design specification for Creditful. It provides detailed requirements, architectural decisions, API contracts, database design, and implementation roadmap to guide development and ensure all components integrate seamlessly.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

Project Name: Creditful
Domain: Financial Technology / Loan Tracking
Primary Users:

1. Borrowers
2. Administrator
3. Public Users (verification only)

Problem Statement: Borrowers need a simple and secure way to track their loan balances and payment history, while also being able to see the loan balances of other borrowers without accessing sensitive information. Admins need a platform to manage loan records. Public users need a way to verify loan existence without seeing sensitive financial information.
Solution: A centralized system accessible via web and mobile applications, built with Spring Boot backend, React web interface, mobile app, and MySQL database. Implement authentication, role-based access, OAuth login, file uploads, email notifications, and optional real-time updates.

2.2 Core User Journeys

Journey 1: Borrower Registration

1. User opens web or mobile app
2. Clicks "Sign Up"
3. Enters email, password, and profile information
4. Account is created
5. User logs in successfully

Journey 2: Borrower Loan Dashboard

1. Borrower logs in
2. Opens dashboard
3. Views loan details: principal, interest, total payable, remaining balance, status
4. Views payment history

Journey 3: Admin Loan Management

1. Admin logs in
2. Creates, updates, deletes, or views loan records
3. Uploads files linked to loans (PDF, image)

4. Manages borrower accounts

Journey 4: Public Loan Verification

1. Public user opens verification page
2. Enters borrower name or ID
3. System checks database
4. Displays: loan existence and status (ACTIVE/COMPLETED)
5. Does not display sensitive financial information

2.3 Feature List (MoSCoW)

MUST HAVE

1. Authentication: register, login, logout, JWT, BCrypt, /me endpoint, protected routes
2. Role-based access control: Admin vs Borrower
3. Loan CRUD operations with validation
4. Borrower dashboard: own loans + other borrowers' loan balances/status
5. Public loan verification
6. Google OAuth login
7. File upload linked to loans
8. Email notifications
9. RESTful API integration

SHOULD HAVE

1. Search borrowers by name or ID
2. Basic input validation
3. Responsive web/mobile design

COULD HAVE

1. Simple dashboard summary
2. Download payment history

WON'T HAVE

1. Credit score integration
2. Advanced analytics dashboard

2.4 Detailed Feature Specifications

Feature: User Authentication

- **Screens:** Registration, Login, Forgot Password
- **Fields:** Email, Password, Confirm Password
- **Validation**: Email format, password strength, uniqueness
- **API Endpoints:** POST /auth/register, POST /auth/login, POST /auth/logout, GET /auth/me
- **Security:** JWT tokens, password hashing with bcrypt, protected routes

Feature: Loan Dashboard

- **Screens:** Loan Listing, Loan Detail
- **Display:** Table layout, showing principal, interest, total payable, remaining balance, status
- **Search:** By borrower name or loan ID
- **API Endpoints:** GET /loans/my-loans, GET /loans/others, GET /loans/verify?borrowerId={id}
- **Admin Functions:** POST /loans, PUT /loans/{id}, DELETE /loans/{id}

Feature: Payment History

- **Screens:** Payment History
- **Fields:** Payment ID, Loan ID, Amount, Payment Date, Updated Remaining Balance
- **Validation:** Amount must be positive, date must be valid
- **API Endpoints:** GET /payments/my-payments, POST /payments
- **Persistence:** Database storage per user

Feature: File Upload

- **Screens:** Loan Detail
- **Fields:** File Name, File Type, Loan ID
- **Validation:** File type allowed (PDF, JPG, PNG), size limit
- **API Endpoints:** POST /files/upload, GET /files/{id}
- **Persistence:** Stored on server + linked to database record

Feature: Admin Panel

- **Screens:** Borrower Management, Loan Management, Payment Overview
- **Fields:** Admin ID, Borrower ID, Loan ID, Payment ID
- **Functions:** Add/edit/delete loans, view borrowers, view payments

- **Access Control:** Admin role required
- **API Endpoints:** Admin-prefixed endpoints with role validation

Feature: Google OAuth Login

- **Screens:** Login Page
- **Fields:** Google Email, Name, OAuth Token
- **Validation:** Verify token, uniqueness in database
- **API Endpoints:** GET /auth/oauth/google
- **Functions:** Link/update user in database, generate JWT

Feature: Email Notifications

- **Screens:** N/A (system triggered)
- **Fields:** Recipient Email, Subject, Body
- **Functions:** Send welcome email, loan creation notification, payment notification
- **API Endpoints:** N/A (handled server-side)

2.5 Acceptance Criteria

AC-1: Successful User Registration

- ➢ Given I am a new user
- ➢ When I enter valid email and strong password
- ➢ And confirm password matches
- ➢ And click "Create Account"
- ➢ Then my account should be created
- ➢ And I should be automatically logged in
- ➢ And redirected to the dashboard

AC-2: Borrower Loan Dashboard

- ➢ Given I am logged in as a borrower
- ➢ When I view my loan dashboard
- ➢ Then I should see all my loans with principal, interest, total payable, remaining balance, and status
- ➢ And I should see other borrowers' loan balances/status (limited info only)

AC-3: Admin Loan Management

- ➢ Given I am logged in as an administrator

- ➢ When I add a new loan with valid details
- ➢ And save the loan
- ➢ Then the loan should appear in the borrower dashboard
- ➢ And be available for management by the admin

AC-4: Payment Recording

- ➢ Given I am logged in as a borrower
- ➢ When I make a payment
- ➢ Then the payment should be recorded
- ➢ And my remaining balance should update

AC-5: File Upload

- ➢ Given I am logged in
- ➢ When I upload a file linked to a loan
- ➢ Then the file should be stored on the server
- ➢ And linked to the correct loan record

AC-6: Google OAuth Login

- ➢ Given I select "Login with Google"
- ➢ When authentication succeeds
- ➢ Then my account should be linked to the database
- ➢ And I should receive a JWT for access

AC-7: Email Notifications

- ➢ Given a new account or payment occurs
- ➢ When the system triggers email
- ➢ Then the user should receive a welcome or notification email

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance Requirements

- The system should respond to user requests within 2–4 seconds under normal usage.
- Loan dashboard pages should load without noticeable delay on a stable internet connection.

- The mobile application should open and display the login screen within a few seconds.
- The system should handle at least 50 concurrent users during testing.
- Loan balance updates should be reflected immediately after payment is recorded.

3.2 Security Requirements

- All communication between client and server must be used by HTTPS.
- The system must implement JWT authentication for protected routes.
- Passwords must be hashed using BCrypt before being stored in the database.
- The system must implement role-based access control (Admin and Borrower).
- Admin-only features (such as creating or deleting loans) must be restricted to users with the admin role.
- The system must use prepared statements or JPA to prevent SQL injection.
- Password fields must never be returned in API responses.

3.3 Compatibility Requirements

- **Web Browsers:** Google Chrome, Mozilla Firefox, and Microsoft Edge (latest stable versions)
- **Android:** API Level 24+ (Android 7.0 and above)
- **Screen Sizes:** Mobile devices (360px and above) and Desktop screens (1024px and above)
- **Operating Systems:** Windows 10+, macOS 10.15+, and Linux-based systems (e.g., Ubuntu 20.04+)

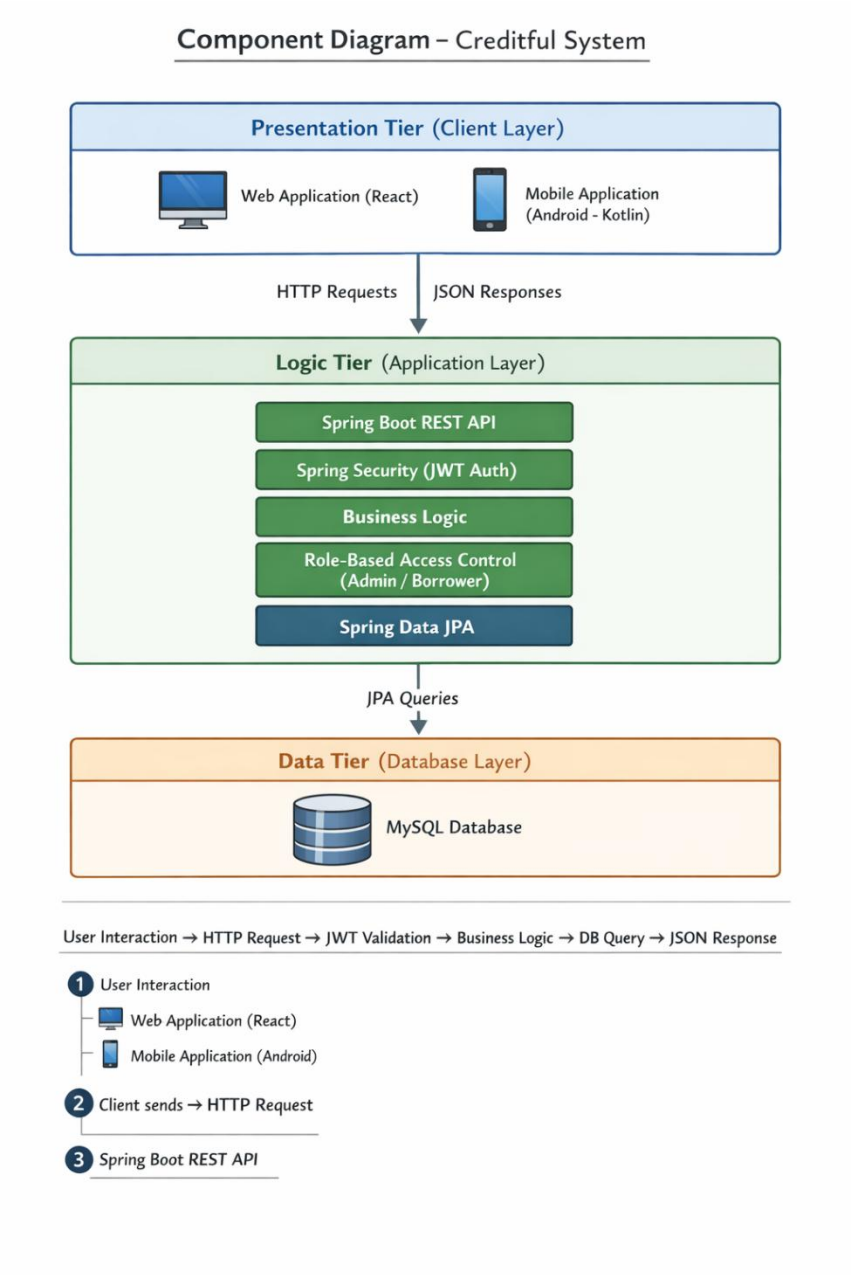
3.4 Usability Requirements

- Users must be able to register and log in without technical assistance.
- Borrowers must clearly see their loan balance and remaining amount on the dashboard.
- Navigation should remain simple and consistent across pages.
- The system should display clear error messages when invalid input is entered.
- Forms should validate required fields before submission.

4.0 SYSTEM ARCHITECTURE

4.1 Component Diagram

Note: This should be a component diagram



Technology Stack:

- **Backend:** Java 17, Spring Boot 3.x, Spring Security, Spring Data JPA
- **Database:** MySQL 8+, MySQL Workbench (schema and table management)

- **Web Frontend:** React 18, JavaScript, Axios
- **Mobile: K**otlin, Android SDK, Retrofit
- **Build Tools:** Maven (Backend), npm (Web), Gradle (Android)
- **Deployment:** Localhost development environment; deployable to VPS or cloud hosting (Backend), static hosting (Web), APK build (Mobile)

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

Base URL	https://[server_hostname]:[port]/api/v1
Format	JSON for all requests/responses
Authentication	Bearer token (JWT) in Authorization header for protected routes

Response Structure	{
	"success": boolean,
	"data": object|null,
	"error": {
	"code": string,
	"message": string,
	"details": object|null
	},
	"timestamp": string
	}

5.2 Endpoint Specifications

Authentication Endpoints

User Registration

Description	Register a new borrower or admin
API URL	/auth/register
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	None

Request Payload	{
	“email”: <email>,
	“password”: <password>,
	“firstname”: <firstname>,
	“lastname”: <lastname>,
	}

Response Structure	{
	"success": boolean,
	"data": {

	user {
	“email”: <email>,
	“firstname”: <firstname>,
	“lastname”: <lastname>,
	},
	“accessToken”: <token>,
	“refreshToken”: <token>,
	},
	"error": {
	"code": string,
	"message": string,
	"details": object|null
	},
	"timestamp": string
	}

User Login

Description	Authenticate existing users and return JWT
API URL	/auth/login
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	None

Request Payload	{
	“email”: <email>,
	“password”: <password>,
	}

Response Structure	{
	"success": boolean,
	"data": {
	“user”: {
	“email”: <email>,
	“firstname”: <firstname>,
	“lastname”: <lastname>,
	“role”: <role>,
	},
	“accessToken”: <token>,
	“refreshToken”: <token>,
	},
	"error": {
	"code": string,
	"message": string,
	"details": object|null
	},
	"timestamp": string
	}

Loan Management Endpoints (Examples)

Get Borrower Loan Balance

Description	Retrieve current loan balance and history for a borrower
API URL	/loans/me
HTTP Request Method	GET
Format	JSON for all requests/responses
Authentication	Required (JWT)
Request Payload	None

Response Structure	{
	"success": boolean,
	"data": {
	"loans": [
	{
	"loanId": <loans.id>,
	"amountBorrowed": <amount_borrowed>,
	"amountRemaining": <amount_remaining>,
	"status": <status>
	}
	]
	},
	"error": {
	"code": string,
	"message": string,
	"details": object|null
	},
	"timestamp": string
	}

Create Loan (Admin Only)

Description	Admin creates a loan record for a borrower
API URL	/loans
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	Required (JWT + Admin role)

Request Payload	{
	"borrowerId": <borrower_id>,
	"amount": <amount_borrowed>
	}

Response Structure	{

	"success": boolean,
	"data": {
	"loanId": <loans.id>,
	"borrowerId": <borrower_id>,
	"amountBorrowed": <amount_borrowed>,
	"amountRemaining": <amount_remaining>,
	"status": <status>
	},
	"error": {
	"code": string,
	"message": string,
	"details": object|null
	},
	"timestamp": string
	}

5.3 Error Handling

HTTP Status Codes

- 200 OK - Successful request
- 201 Created - Resource created
- 400 Bad Request - Invalid input
- 401 Unauthorized - Authentication required/failed
- 403 Forbidden - Insufficient permissions
- 404 Not Found - Resource doesn't exist
- 409 Conflict - Duplicate resource
- 500 Internal Server Error - Server error

Error Code Examples

Example 1	json
	{
	"success": false,
	"data": null,
	"error": {
	"code": "AUTH-001",
	"message": "Invalid credentials",
	"details": "Email or password is incorrect"
	},
	"timestamp": "2024-01-28T10:30:00Z"
	}

Example 2	{
	"success": false,
	"data": null,
	"error": {
	"code": "VALID-001",
	"message": "Validation failed",
	"details": {
	"email": "Email is required",
	"password": "Must be at least 8 characters"
	}
	},
	"timestamp": "2024-01-28T10:30:00Z"
	}

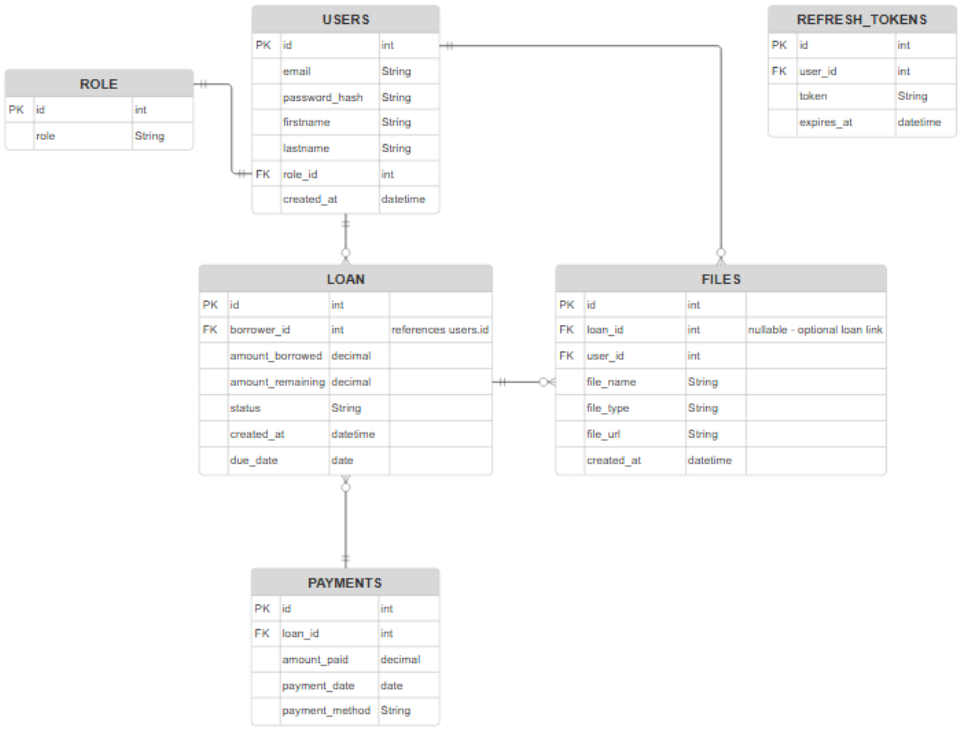
Common Error Codes

- AUTH-001: Invalid credentials
- AUTH-002: Token expired
- AUTH-003: Insufficient permissions
- VALID-001: Validation failed
- DB-001: Resource not found
- DB-002: Duplicate entry
- BUSINESS-001: Loan limit exceeded
- SYSTEM-001: Internal server error

6.0 DATABASE DESIGN

6.1 Entity Relationship Diagram

Note: This should be an ERD (Database Schema)



Detailed Relationships:

- **One-to-One: User** ↔ Role (Each user has exactly one role – Admin or Borrower)
- **One-to-Many: User** → Loans (A user can have multiple loans)
- **One-to-Many: Loan** → Payments (A loan can have multiple payment records)
- **One-to-Many: User** → Files (User can upload multiple documents linked to loans)
- **Many-to-One: Payments** → Loan (Each payment is linked to a loan)

Key Tables:

1. **users –** User accounts and authentication
2. **roles –** Defines Admin and Borrower roles
3. **loans –** Loan records for borrowers
4. **payments –** Records of loan repayments
5. **files –** Uploaded documents linked to loans
6. **refresh_tokens –** JWT refresh tokens

Table Structure Summary:

- **users:** id, email, password_hash, firstname, lastname, role_id, created_at
- **roles:** id, role (Admin / Borrower)

- **loans:** id, borrower_id, amount_borrowed, amount_remaining, status, created_at, due_date
- **payments:** id, loan_id, amount_paid, payment_date, payment_method
- **files:** id, loan_id, user_id, file_name, file_type, file_url, uploaded_at
- **refresh_tokens:** id, user_id, token, expires_at

7.0 UI/UX DESIGN

7.1 Web Application Wireframes

Note: This should be wireframes from Figma

Homepage

Product Detail Page

Shopping Cart Page

Checkout Page

Admin Dashboard

7.2 Mobile Application Wireframes

Note: This should be wireframes from Figma

Bottom Navigation

Home Screen

Product Detail Screen

Cart Screen

Checkout Flow

Mobile-Specific Features:

Design System:

8.0 PLAN

8.1 Project Timeline

Phase 1: Planning & Design (Week 1-2)

Week 1: Requirements & Architecture

Day 1-2: Project setup and documentation

Day 3-4: Complete Functional & Non-Functional Requirements

Day 5-7: System architecture outline

Week 2: Detailed Design

Day 1-2: API endpoints specification

Day 3-4: Database schema design in MySQL Workbench

Day 5-6: UI wireframes for web and mobile

Day 7: Finalize implementation plan

Phase 2: Backend Development (Week 3-4)

Week 3: Foundation

Day 1: Spring Boot project setup with dependencies

Day 2: Database configuration and entity creation

Day 3: JWT authentication and BCrypt password hashing

Day 4: User registration, login, logout endpoints

Day 5: Role-based access control

Week 4: Core Features

Day 1: Loan entity CRUD (Admin)

Day 2: Borrower dashboard endpoints (loan balance, history)

Day 3: Payment recording and balance update endpoints

Day 4: File upload endpoints (linked to loans)

Day 5: Testing and debugging backend

Phase 3: Web Application (Week 5-6)

Week 5: Frontend Foundation

Day 1: React project setup

Day 2: Authentication pages (login, register)

Day 3: Loan dashboard for borrowers

Day 4: Admin panel pages

Day 5: Basic navigation and page linking

Week 6: Complete Web Features

Day 1: Payment history page

Day 2: Loan creation/editing for Admin

Day 3: File upload UI integration

Day 4: Responsive design adjustments

Day 5: Connect frontend with backend APIs and testing

Phase 4: Mobile Application (Week 7-8)

Week 7: Android Foundation

Day 1: Android Studio setup and project structure

Day 2: Authentication screens (login, register)

Day 3: Borrower loan dashboard

Day 4: Admin loan management screens

Day 5: API service layer with Retrofit

Week 8: Complete Mobile App

Day 1: Payment recording and history

Day 2: File upload functionality

Day 3: UI polish and minor animations

Day 4: Testing on emulator/device

Day 5: APK generation and documentation

Phase 5: Integration & Deployment (Week 9-10)

Week 9: Integration Testing

Day 1: End-to-end testing across web and mobile

Day 2: Bug fixing

Day 3: Security review

Day 4: Performance testing

Day 5: Documentation updates

Week 10: Deployment

Day 1: Backend deployment (local VPS or cloud)

Day 2: Web application deployment (Vercel/Netlify)

Day 3: Mobile APK distribution

Day 4: Final testing across all platforms

Day 5: Project submission

Milestones:

- **M1 (End Week 2):** All design documents complete
- **M2 (End Week 4):** Backend API fully functional
- **M3 (End Week 6):** Web application complete
- **M4 (End Week 8):** Mobile application complete
- **M5 (End Week 10):** Full system deployed and integrated

Critical Path:

1. Authentication system (Week 3)
2. Loan CRUD and borrower dashboard (Week 4)
3. Payment recording (Week 4-5)
4. Admin panel integration (Week 6-7)
5. Cross-platform testing (Week 9)

Risk Mitigation:

Start with a simple working version of each feature

Test integration points early and often

Keep backup copies of working code

Focus on core functionality before enhancements